



UNIVERSIDAD COMPLUTENSE MADRID

DISEÑO AUTOMÁTICO DE SISTEMAS

LABORATORIO 2: Cronómetro

Daniel Portuondo Rodríguez

Declaro que este trabajo es resultado de mi propio esfuerzo, que todas las fuentes consultadas han sido citadas adecuadamente y que no he incurrido en plagio ni en ninguna otra forma de deshonestidad académica.

2. Diseño

2.1. Especificación del diseño

- **Descripción funcional clara del sistema.**

El sistema se trata de una cronómetro con start/stop, lap, clear y reset, generando una salida de 16 bits que es mostrada en formato binario por los leds de la placa de prototipado Xilinx Basys 3.

- **Entradas y salidas (tabla con nombre, ancho y descripción).**

Nombre	Tipo	Ancho (bits)	Descripción
clk	Entrada	1	Señal de reloj generada por un oscilador externo.
clear	Entrada	1b	Señal que limpia el contenido de los registros del sistema.
reset	Entrada	1b	Señal que reinicia el sistema, limpiando los registros y deteniendo el funcionamiento hasta que se vuelva a pulsar startStop.
startStop	Entrada	1b	Señal que inicia y para el cronómetro.
lap	Entrada	1b	Señal que tras ser activada, congela los leds, pero sin parar la cuenta del cronómetro.
leds	Salida	16b	- leds(15): cifra menos significativa de la cuenta de las decenas de segundo - leds(14-8): '0' - leds(7-4): Contador de la cifra izquierda de los segundos - leds(3-0): Contador de la cifra derecha de los segundos

• **Requisitos funcionales y restricciones.**

- **Requisito funcional 1.0:** El sistema ha de poder llevar la cuenta de los segundos transcurridos desde su activación mediante la señal de start.
- **Requisito funcional 1.1:** El sistema ha de poder ser detenido mediante la señal de stop, sin perder la cuenta
- **Requisito funcional 1.3:** La cuenta ha de poder ser reanudada, sin perder información.
- **Requisito funcional 2.0:** La señal de reset ha de reiniciar el sistema al completo, tal como si se acabara de encender.
- **Requisito funcional 2.1:** La señal de clear ha de limpiar el contenido de los registros, tal como si se acabara de encender el sistema, sin afectar al resto de componentes del mismo.
- **Requisito funcional 3:** La señal de lap ha de detener la lógica de los leds, congelandolos en el estado en el que estuvieran al activarse la señal. Adicionalmente, se conservará la cuenta que se llevase hasta el momento y se iniciará una segunda cuenta independiente de la primera. Si la señal fuera activada de nuevo, se reanudaría la cuenta inicial y su muestra en los leds.
- **Requisito funcional 4.0:** el sistema ha de generar el resultado en formato de señal std_logic_vector de 16 bits, con el fin de ser conectada a 16 leds para mostrar el resultado de manera visual.
- **Requisito funcional 4.1:** El led más significativo ha de parpadear a 5 Hz (encenderse y apagarse 5 veces por segundo).

• **Supuestos adoptados.**

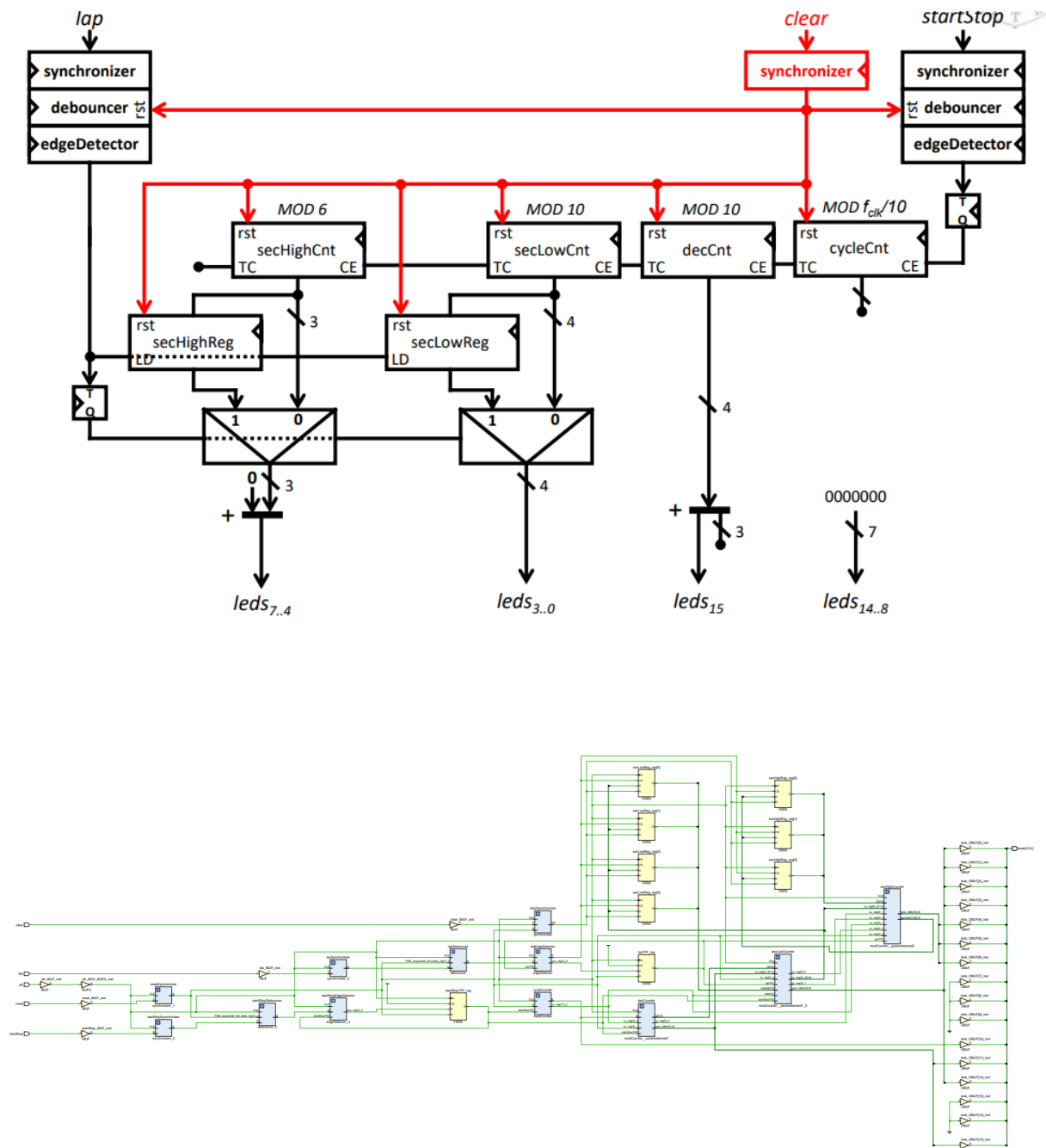
- **Conexión externa:** Se asume que los receptores de la salida del sistema sean descritos en el requisito funcional 4.0
- **Lógica directa sw:** Se ha dado por hecho que los switches son de lógica directa (ofrecen un 1 lógico al ser activados)
- **Lógica directa btnx:** A su vez, el sistema también depende de que los botones sean igualmente de lógica directa.

2.2. Descripción de la arquitectura

Se trata de un sistema basado en lógica secuencial síncrona, organizado como un datapath de contadores en cascada controlado por señales de entrada acondicionadas. El núcleo del diseño es una cadena de cuatro contadores modulares instanciados a partir del componente genérico `modCounter`: un contador de ciclos ($\text{MOD } f_{\text{clk}}/10$), un contador de décimas de segundo ($\text{MOD } 10$), un contador de unidades de segundo ($\text{MOD } 10$) y un contador de decenas de segundo ($\text{MOD } 6$). Cada uno habilita al siguiente a través de su señal de terminal count (`tc`), formando así un cronómetro en BCD módulo 60.

Las tres señales de entrada (`startStop`, `lap`, `clear`) provienen de pulsadores físicos y son asíncronas y ruidosas, por lo que cada una pasa por una cadena de acondicionamiento compuesta por un `synchronizer`, un `debouncer` y un `edgeDetector`, todos ellos módulos genéricos y reutilizables declarados en `common.vhd`. La señal `clear` solo necesita sincronización ya que actúa como reset síncrono directo. La señal `startStop` se convierte en un biestable T (`startStopTFF`) que alterna el estado de marcha/parada del cronómetro habilitando o deshabilitando los contadores. La señal `lap` activa el almacenamiento de la cuenta actual en dos registros (`secHighReg` y `secLowReg`) y congela la visualización mediante otro biestable T (`lapTFF`).

La visualización se realiza a través de dos multiplexores que seleccionan entre la salida de los contadores y la de los registros de lap según el estado de `lapTFF`. Los 8 leds menos significativos muestran la cuenta en BCD, el led 15 parpadea a 5 Hz generado a partir del terminal count del contador de décimas, y los leds 14 a 8 permanecen a cero. Adjunto el diagrama RTL proporcionado por el guión de la práctica y el esquemático generado por Vivado tras la implementación.



2.3. Decisiones de diseño

2.3.1. Dominios de reloj

Sólo hay un dominio de reloj: sysClk, con periodo 10ns y frecuencia 100MHz

2.3.2. Dominios de reset

Hay un único dominio de reset.

Además usando el comando "get_pins -hierarchical -filter {IS_RESET == 1}", se puede comprobar todos los pines de reset del sistema, y si alguno de ellos es asíncrono.

3. Resultados de implementación física

3.1. Utilización de recursos

Tipo	Usados	%Uso
LUTs	111	0.53%
FF	106	0.25%
BRAM	0	0%
DSP	0	0%
IOs	21	19.81%

3.2. Resultados de timing

Podemos ver que todos los timing constraints definidos se cumplen:

- **WNS:** 5.414 ns
- **TNS:** 0 ns

Tenemos 12 caminos sin restricciones:

- **NONE to sysClk:** El sistema cuenta con 4 caminos sin restricciones de este tipo (las señales asíncronas reset, clear, lap y startstop). El que mayor tiempo de hold presenta es el que une reset con resetSynchronizer/aux_reg[0]/D (la entrada del sincronizador), de 2.409ns.
El que mayor tiempo de setup presenta es también el que une reset con resetSynchronizer/aux_reg[0]/D, con un delay de 6.115ns.
- **sysClk to NONE:** El sistema cuenta con 8 caminos sin restricciones de este tipo (las salidas hacia los leds). El que mayor tiempo de setup presenta es el que une secHighCounter/cs_reg[2]/C con leds[6], de 11.032ns.
El que mayor tiempo de hold presenta es el que une secHighReg_reg[1]/C con leds[5], con un delay de 4.011ns.

3.3. Consumo de potencia

- **Estimación estática**

0.072W

- **Estimación dinámica**

0.009W (11% Clocks, 10% Signals, 9% Logic, 70% IO)

4. Análisis crítico de resultados

El sistema implementado cumple correctamente todos los requisitos funcionales especificados, con una utilización de recursos extraordinariamente reducida.

El timing es holgado, con un WNS de 5,414 ns sobre un período de reloj de 10 ns, lo que implica que el camino crítico consume aproximadamente 4,6 ns. Hay por tanto margen suficiente para aumentar la frecuencia de operación sin violar las restricciones de setup y hold.

Los 12 caminos sin restricciones reportados por Vivado tienen sentido y son los esperados, no son ningún tipo de motivo para preocupación.

El consumo de potencia es mínimo, con tan solo 0,081 W totales, siendo el 70% atribuible a las IOs. Esto refleja que el cuello de botella no es la lógica interna sino la interfaz física con los LEDs.

Una posible mejora sería que se mostrase la cuenta de las diferentes laps, en vez de congelarse la visualización, además de poder tener más de 2 laps.

5. Dificultades encontradas y soluciones adoptadas

Parpadeo Led: En el esquema proporcionado por el guión de la práctica, se especificaba que el led(15) había que conectarlo con el bit más significativo del contador de las décimas de segundo, sin embargo esto generaba un comportamiento diferente al deseado, encendiéndose únicamente una vez por segundo en vez de las 5 deseadas. Para solucionarlo, bastó con conectar el bit menos significativo del contador.

6. Anexos

- **Nombre del archivo zip con el proyecto Vivado**

Daniel_Portuondo_lab2.zip

- **Ficheros de constraints**

DAS_labs/source/lab2/lab2.xdc

- **Scripts de síntesis**

DAS_labs/projects/lab2/lab2.runs/synth_1/lab2.tcl

- **Versiones de herramientas y lenguajes**

Vivado v2023.1 y VHDL

- **Dispositivo FPGA utilizado**

Digilent Basys 3 Artix-7